

Aufgabenstellung: Abstrakte Klassen in Java

(Klassen „Künstler“, „Maler“, „Sänger“ sowie Arbeiten mit abstrakten Oberklassen und konkreten Unterklassen)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das eine einfache Klassenstruktur mit einer **abstrakten Oberklasse** und zwei **konkreten Unterklassen** modelliert.

Sie lernen dabei,

- wie abstrakte Klassen in Java verwendet werden,
- wie eine abstrakte Methode in einer Oberklasse vorgegeben wird,
- wie konkrete Unterklassen diese Methode unterschiedlich umsetzen,
- wie gemeinsame und spezialisierte Rollen in einer Klassenhierarchie dargestellt werden,
- wie ein Testprogramm die Beziehungen zwischen Oberklasse und Unterklassen sichtbar macht.

Der Fokus liegt auf dem Verständnis, dass eine abstrakte Klasse als allgemeines Modell dient, von dem keine direkten Objekte erzeugt

werden, während Unterklassen dieses Modell durch eigenes Verhalten konkret ausfüllen.

Fachlicher Hintergrund

In der objektorientierten Programmierung werden Klassen häufig in Beziehung zueinander gesetzt.

Eine wichtige Möglichkeit ist die Verwendung von **abstrakten Klassen**.

Eine abstrakte Klasse beschreibt ein allgemeines Grundmodell. Sie kann Methoden vorgeben, die in Unterklassen konkret umgesetzt werden müssen.

In dieser Aufgabe wird die Klasse `Künstler` als allgemeine abstrakte Oberklasse verwendet.

Davon leiten sich die Klassen `Maler` und `Sänger` ab.

Typische Beziehungen in dieser Aufgabe sind:

- ein `Maler` ist ein `Künstler`,
- ein `Sänger` ist ein `Künstler`,
- beide besitzen die gemeinsame Eigenschaft, Kunst zu erstellen,
- beide unterscheiden sich aber in der konkreten Umsetzung.

Damit eignet sich dieses Beispiel gut, um den Unterschied zwischen einer **abstrakten Oberklasse** und **konkreten Unterklassen** zu verdeutlichen.

Vorgegebene Klassenstruktur

Im Package `abstrakt.template` sollen folgende Klassen entwickelt und verwendet werden:

- `Künstler`
- `Maler`
- `Sänger`
- eine Klasse mit `main`-Methode zum Testen

Die Klassen haben dabei folgende Rollen:

- `Künstler` ist die **abstrakte Oberklasse**,
- `Maler` und `Sänger` sind **konkrete Unterklassen** von `Künstler`,
- die `main`-Methode dient zum Testen des Programms.

Zusätzlich soll ein **Klassendiagramm** zur Veranschaulichung der

Klassenstruktur gezeichnet werden.

Aufgabe 1: Entwicklung der abstrakten Klasse „Künstler“

Ziel

Sie entwickeln die Klasse `Künstler` als abstrakte Oberklasse der Klassenstruktur.

Zu entwickelnde Klassenstruktur

Die Klasse `Künstler` soll eine abstrakte Methode enthalten, die von Unterklassen umgesetzt werden muss.

Beispielhaft:

```
public abstract class Künstler
{
    public abstract void kunstErstellen();
}
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Künstler` so, dass:

- sie als abstrakte Oberklasse dient,
- sie nicht direkt instanziiert werden soll,
- sie die Methode `kunstErstellen()` als abstrakte Methode vorgibt.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `abstract`,
- eine abstrakte Methode `kunstErstellen()`,
- eine sinnvolle Oberklasse für mehrere spezialisierte Unterklassen.

Ziel dieser Aufgabe

Sie sollen verstehen,

- welche Aufgabe eine abstrakte Oberklasse hat,
- warum eine abstrakte Klasse nicht direkt zur Objekterzeugung gedacht ist,
- wie gemeinsames Verhalten in allgemeiner Form vorgegeben werden kann.

Aufgabe 2: Entwicklung der Klasse „Maler“

Ziel

Sie entwickeln die Klasse `Maler` als konkrete Unterklasse von `Künstler`.

Zu entwickelnde Klassenstruktur

```
public class Maler extends Künstler
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Maler` so, dass:

- sie alle allgemeinen Vorgaben der Klasse `Künstler` übernimmt,
- sie die Methode `kunstErstellen()` konkret umsetzt,
- bei einem Aufruf der Methode die Ausgabe

```
Ich male ein Bild
```

auf der Konsole erscheint.

Technische Anforderungen

Verwenden Sie:

- `extends Künstler`,
 - eine Überschreibung der Methode `kunstErstellen()`,
 - eine passende Konsolenausgabe.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine konkrete Unterklasse aus einer abstrakten Oberklasse entsteht,
 - wie eine abstrakte Methode in einer Unterklasse umgesetzt wird,
 - wie Spezialisierung durch Methodenüberschreibung erfolgt.
-

Aufgabe 3: Entwicklung der Klasse „Sänger“

Ziel

Sie entwickeln die Klasse `Sänger` als weitere konkrete Unterklasse von `Künstler`.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Sänger` so, dass:

- sie die Vorgaben der Klasse `Künstler` übernimmt,
- sie die Methode `kunstErstellen()` ebenfalls konkret umsetzt,
- bei einem Aufruf der Methode die Ausgabe

`Ich singe einen Song`

auf der Konsole erscheint.

Technische Anforderungen

Verwenden Sie:

- `extends Künstler`,
- eine eigene Implementierung der Methode `kunstErstellen()`,
- eine passende Konsolenausgabe.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass mehrere konkrete Unterklassen aus derselben abstrakten Oberklasse entstehen können,
 - wie dieselbe Methode in verschiedenen Unterklassen unterschiedlich umgesetzt wird,
 - wie unterschiedliches Verhalten modelliert werden kann.
-

Aufgabe 4: Analyse der Methodenüberschreibung

Ziel

Sie analysieren, wie eine abstrakte Methode in verschiedenen Unterklassen konkret umgesetzt wird.

Anforderungen

Funktionale Anforderungen

Untersuchen und entwickeln Sie die Klassen so, dass:

- die Methode `kunstErstellen()` in der Oberklasse nur vorgegeben wird,
- die Unterklassen `Ma1er` und `Sanger` diese Methode jeweils unterschiedlich umsetzen,
- bei der Ausgabe deutlich wird, welche konkrete Kunstlerklasse aktiv

ist.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Methoden,
- Methodenüberschreibung in Unterklassen,
- unterschiedliche Konsolenausgaben bei gleichem Methodennamen.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was eine abstrakte Methode ist,
- warum Unterklassen diese Methode umsetzen müssen,
- wie dadurch gemeinsames Grundmodell und konkretes Verhalten zusammenwirken.

Aufgabe 5: Entwicklung der Testklasse mit main-Methode

Ziel

Sie entwickeln ein Testprogramm, das die Klassenstruktur praktisch demonstriert.

Zu entwickelnde Klasse

Die Testklasse soll eine `main`-Methode enthalten.

Anforderungen

Funktionale Anforderungen

Das Testprogramm soll:

1. ein Objekt der Klasse `Ma1er` erzeugen,
2. ein Objekt der Klasse `Sanger` erzeugen,
3. fur beide Objekte die Methode `kunstErstellen()` aufrufen,
4. die Unterschiede der Ausgaben auf der Konsole sichtbar machen.

Technische Anforderungen

Verwenden Sie:

- eine `main`-Methode,
- mehrere Objekterzeugungen,

- Methodenaufrufe auf Objekten verschiedener Klassen,
 - sinnvolle Konsolenausgaben.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie konkrete Unterklassen im Programm verwendet werden,
 - wie eine abstrakte Klassenstruktur getestet wird,
 - wie sich unterschiedliches Verhalten in der Ausgabe zeigt.
-

Aufgabe 6: Zeichnen eines Klassendiagramms

Ziel

Sie stellen die Klassenstruktur grafisch dar.

Anforderungen

Erstellen Sie ein Klassendiagramm, das zeigt:

- die abstrakte Oberklasse `Künstler`,
 - die beiden Unterklassen `Maler` und `Sänger`,
 - die Vererbungsbeziehungen zwischen den Klassen,
 - die Methode `kunstErstellen()`.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Klassenhierarchien grafisch dargestellt werden,
 - wie Ober- und Unterklassen im Diagramm erkennbar sind,
 - wie Methoden in einem Klassendiagramm eingetragen werden.
-

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was eine abstrakte Klasse in Java ist,
- den Unterschied zwischen abstrakter Oberklasse und konkreter Unterklasse beschreiben,

- Klassen mit `extends` modellieren,
 - abstrakte Methoden in Unterklassen überschreiben,
 - eine einfache Klassenhierarchie in einem Testprogramm anwenden,
 - ein Klassendiagramm zu einer Vererbungsstruktur zeichnen.
-

Didaktische Einordnung

Diese Aufgabe erweitert grundlegende Java-Kenntnisse um das Konzept der **abstrakten Klassen** und der **Methodenüberschreibung**.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Methoden
- Vererbung
- `extends`
- abstrakte Klassen
- abstrakte Methoden
- Methodenüberschreibung
- Oberklassen und Unterklassen
- UML-Klassendiagramme

Die Aufgabe zeigt, dass abstrakte Klassen eine gemeinsame Grundlage für verschiedene konkrete Klassen bilden können.

Sie bildet eine wichtige Grundlage für weiterführende Themen wie:

- Polymorphie
- dynamische Bindung
- Interfaces
- objektorientierter Softwareentwurf

Reflexions- und Kontrollfragen

Kontrollfragen zu abstrakten Klassen

1. Was ist eine abstrakte Klasse in Java?
2. Warum kann `Künstler` nicht direkt instanziiert werden?
3. Welche Aufgabe hat die Oberklasse `Künstler`?
4. Welche Aufgabe haben die Unterklassen `Maler` und `Sänger`?

Kontrollfragen zur Klassenstruktur

5. Warum erben `Maler` und `Sänger` von `Künstler`?
 6. Welche Gemeinsamkeit besitzen beide Klassen?
 7. Worin unterscheiden sich `Maler` und `Sänger`?
 8. Warum ist hier eine abstrakte Oberklasse sinnvoll?
-

Kontrollfragen zur Programmlogik

9. Was ist eine abstrakte Methode?
 10. Warum muss `kunstErstellen()` in den Unterklassen umgesetzt werden?
 11. Was zeigt die Ausgabe des Programms?
 12. Welche Vorteile bietet diese Modellierung?
-